

MIP_PROMPT_LIBRARY_V2_PART2_PARSING.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 2 - Parsing Prompts

Version: 2.1

Status: Enhanced Enterprise Architecture

Purpose

Discovery tells us what exists.

Parsing transforms source code, control artifacts, runtime definitions, and operational metadata into structured intelligence.

The parser layer is the foundation of MIP.

Without reliable parsers:

- Metadata becomes unreliable
- Knowledge graphs become inaccurate
- Impact analysis becomes dangerous
- Modernization recommendations become invalid
- Business capability mapping becomes incomplete
- Runtime architecture remains hidden

The goal is not code conversion.

The goal is complete metadata extraction, relationship discovery, lineage generation, and architectural intelligence.

Parser Engineering Principles

Every parser must be:

Deterministic

Same input must always produce same output.

Explainable

Every extracted fact must point to source evidence.

Extensible

Support future language variants and platform-specific extensions.

Fault Tolerant

Continue processing even when source contains errors.

Observable

Provide detailed extraction metrics.

Incremental

Support re-parsing only changed artifacts.

Graph-Aware

Generate metadata optimized for knowledge graph construction.

Enterprise-Scale

Support millions of source artifacts.

Cross-Artifact Intelligent

Correlate information across COBOL, JCL, Copybooks, DB2, VSAM, CICS, MQ, IMS, Scheduler Definitions, and Operational Metadata.

Advanced Parsing Philosophy

Traditional parsers extract syntax.

MIP parsers extract intelligence.

Every parser should operate at multiple levels:

Level 1 - Syntax Parsing

Extract language constructs.

Level 2 - Semantic Parsing

Understand relationships between constructs.

Level 3 - Dependency Parsing

Identify dependencies across artifacts.

Level 4 - Runtime Parsing

Infer execution behavior.

Level 5 - Architectural Parsing

Identify applications, domains, services, and business capabilities.

Level 6 - Modernization Parsing

Identify API candidates, service candidates, reusable assets, and modernization opportunities.

Advanced Enterprise Parsing Techniques

To support any mainframe environment, MIP should implement additional parsing strategies.

Multi-Pass Parsing

Pass 1:

- Syntax extraction

Pass 2:

- Dependency extraction

Pass 3:

- Cross-reference resolution

Pass 4:

- Runtime relationship discovery

Pass 5:

- Graph enrichment

This approach significantly improves accuracy.

Cross-Repository Correlation Parsing

Many dependencies are not visible within a single source file.

Correlate:

- COBOL ↔ Copybooks
- COBOL ↔ JCL
- COBOL ↔ DB2
- COBOL ↔ VSAM
- COBOL ↔ MQ
- COBOL ↔ CICS
- COBOL ↔ IMS
- JCL ↔ Scheduler
- JCL ↔ Procedures
- Programs ↔ Operational Assets

This creates enterprise-wide visibility.

Symbol Resolution Engine

Resolve:

- COPY statements
- CALL targets
- Dynamic CALLs
- SQL host variables
- Dataset aliases
- Procedure references
- Scheduler references

Many modernization tools fail because they cannot resolve indirect references.

Dynamic Call Discovery

Static CALL statements are easy.

Dynamic CALL statements are difficult.

Examples:

```
CALL WS-PROGRAM-NAME
```

Parser should identify:

- Potential targets
- Variable origins
- Configuration-driven targets
- Runtime resolution paths

Generate confidence scores.

Data Flow Parsing

Track:

- Source fields
- Intermediate fields
- Target fields

Identify:

- Field movements
- Transformations
- Aggregations
- Calculations

This becomes the foundation for lineage and impact analysis.

Control Flow Parsing

Extract:

- PERFORM chains
- GO TO paths
- Conditional branches
- Error handling paths
- Exit paths

Generate execution flow metadata.

Business Rule Extraction

Without interpreting business meaning, identify:

- Decision points
- Validation logic
- Threshold checks
- Eligibility rules
- Calculation sections

Tag these as potential business-rule candidates.

Runtime Behavior Inference

Infer:

- Batch execution paths

- Online transaction paths
- Data movement paths
- Integration paths

Generate confidence-ranked runtime models.

Pattern Recognition Parsing

Identify:

- CRUD patterns
- Batch patterns
- ETL patterns
- Reporting patterns
- Service patterns
- Utility patterns

Classify programs automatically.

AI-Assisted Metadata Enrichment

After deterministic parsing:

Use AI to identify:

- Business capability candidates
- Domain candidates
- Service candidates
- API candidates
- Modernization opportunities

All AI-generated metadata must be tagged separately from deterministic metadata.

Prompt 06

COBOL Parser Architecture

File

prompts/parsing/cobol_parser.md

Purpose

Design a production-grade COBOL parser capable of extracting metadata from enterprise COBOL applications.

Context

COBOL programs contain:

- Program definitions
- Business logic
- Program dependencies
- Database access
- File access
- Transaction processing
- Integration logic
- Business rules

The parser must extract metadata without attempting business interpretation.

Inputs

COBOL source files.

Instructions

Design a parser architecture capable of extracting:

Identification Division

- PROGRAM-ID
- AUTHOR
- INSTALLATION
- DATE-WRITTEN
- SECURITY CLASSIFICATIONS
- COMMENTS

Environment Division

- File assignments
- Configuration
- Special names
- Device mappings

Data Division

- Working Storage
- File Section
- Linkage Section
- Local Storage
- Screen Section
- Report Section

Data Structures

- Levels
- PIC Clauses

- REDEFINES
- OCCURS
- OCCURS DEPENDING ON
- VALUE Clauses
- 88-Level Conditions

Procedure Division

- CALL Statements
- Dynamic CALL Statements
- PERFORM Statements
- Paragraphs
- Sections
- GO TO Statements
- EVALUATE Statements
- IF Conditions
- Error Handling Logic

File Operations

- OPEN
- CLOSE
- READ
- WRITE
- REWRITE
- DELETE
- START

Embedded SQL

- Tables
- Views
- Operations
- Cursors
- Host Variables

Integration Metadata

- MQ Calls
- CICS Commands
- IMS Calls
- External Interfaces

Data Flow Metadata

- MOVE Statements
- Field Transformations
- Source-to-Target Mapping

Generate:

- Architecture Diagram
- Modules

- Data Models
- Error Handling Strategy
- Testing Strategy
- Dependency Resolution Strategy

Expected Output

Parser Overview

Component Diagram

Metadata Extraction Flow

Dependency Resolution Design

Data Flow Extraction Design

Error Handling Design

Testing Design

Constraints

Do not generate implementation code.

Focus on architecture and metadata extraction.

Success Criteria

Parser supports enterprise-scale COBOL repositories and captures both static and runtime-oriented metadata.

Example Usage

Design a parser architecture for a repository containing 10,000+ COBOL programs.

Review Checklist

- PROGRAM-ID extraction supported?
 - Dynamic CALL extraction supported?
 - SQL extraction supported?
 - MQ extraction supported?
 - CICS extraction supported?
 - Data flow extraction supported?
 - Fault tolerance defined?
 - Metadata model aligned?
-

Prompt 07

JCL Parser Architecture

File

prompts/parsing/jcl_parser.md

Purpose

Design a parser for Job Control Language.

Context

JCL defines:

- Execution flows
- Program execution
- Dataset usage
- Batch orchestration
- Operational dependencies

Understanding JCL is critical for root-program identification.

Inputs

JCL source members.

Instructions

Design parser architecture capable of extracting:

Job Metadata

- JOB Statements
- Parameters
- Accounting Information

Execution Metadata

- EXEC Statements
- Programs
- Procedures
- Nested Procedures

Dataset Metadata

- DD Statements
- Inputs
- Outputs

- Temporary Datasets
- GDGs

Scheduling Metadata

- Dependencies
- Conditions
- Return Code Logic

Operational Metadata

- Restart Logic
- Recovery Logic
- Utility Usage

Generate architecture and metadata model.

Expected Output

Parser Design

Metadata Model

Execution Flow Model

Extraction Flow

Error Handling

Constraints

No implementation code.

Success Criteria

Every JCL job can be represented as metadata and execution graphs.

Example Usage

Design a parser for 5,000 production JCL jobs.

Review Checklist

- JOB extraction supported?
- EXEC extraction supported?
- DD extraction supported?
- Dependency extraction supported?
- Procedure expansion supported?
- Restart logic captured?

Prompt 08

Copybook Parser Architecture

File

prompts/parsing/copybook_parser.md

Purpose

Design parser architecture for enterprise copybooks.

Context

Copybooks define shared data structures across thousands of programs.

They often contain the most reusable business knowledge.

Inputs

Copybook source files.

Instructions

Support extraction of:

Structure Metadata

- Levels
- Field Names
- PIC Clauses
- VALUE Clauses

Complex Structures

- REDEFINES
- OCCURS
- OCCURS DEPENDING ON
- Nested Structures

Relationships

- Parent Child Hierarchies
- Reusable Structures
- Shared Usage

Data Intelligence

- Canonical Data Structures
- Common Business Entities

- Reuse Metrics

Generate parser architecture and metadata schema.

Expected Output

Parser Architecture

Metadata Model

Validation Strategy

Complexity Scoring Model

Reuse Analysis Model

Constraints

No business interpretation.

Success Criteria

Complex enterprise copybooks represented consistently.

Example Usage

Design parser architecture for 20,000 copybooks.

Review Checklist

- PIC supported?
- OCCURS supported?
- REDEFINES supported?
- Nested structures supported?
- Reuse analysis supported?

Prompt 09

DB2 SQL Parser Architecture

File

prompts/parsing/db2_sql_parser.md

Purpose

Design parser architecture for embedded DB2 SQL.

Context

SQL statements reveal:

- Data dependencies
- CRUD operations
- Data lineage
- Integration points

Inputs

EXEC SQL blocks.

Instructions

Support extraction of:

Objects

- Tables
- Views
- Aliases
- Synonyms

Operations

- SELECT
- INSERT
- UPDATE
- DELETE
- MERGE

Access Patterns

- Joins
- Predicates
- Cursors
- Subqueries

Relationships

- Program → Table
- Program → View
- Program → Cursor

Data Lineage

- Source Columns
- Target Columns
- Transformation Paths

Generate metadata architecture.

Expected Output

Parser Architecture

Dependency Model

Metadata Schema

Lineage Model

Validation Rules

Constraints

Focus only on metadata extraction.

Success Criteria

All SQL dependencies become graph-ready metadata.

Example Usage

Analyze DB2 usage across an enterprise card-processing application.

Review Checklist

- CRUD extraction supported?
 - Table extraction supported?
 - Join extraction supported?
 - Cursor extraction supported?
 - Column lineage supported?
-

Prompt 10

VSAM Parser Architecture

File

prompts/parsing/vsam_parser.md

Purpose

Design parser architecture for VSAM metadata extraction.

Context

Many mainframe systems rely heavily on VSAM.

VSAM relationships are critical modernization inputs.

Inputs

COBOL source code and VSAM definitions.

Instructions

Support extraction of:

VSAM Objects

- KSDS
- ESDS
- RRDS
- LDS

Access Methods

- READ
- WRITE
- REWRITE
- DELETE
- START

Relationships

- Program → VSAM
- Job → VSAM

Data Flow

- Record Structures
- Key Usage
- Access Patterns

Generate architecture and metadata model.

Expected Output

Parser Design

Metadata Schema

Relationship Model

Data Flow Model

Validation Rules

Constraints

Do not infer business semantics.

Success Criteria

VSAM usage becomes traceable and graph-ready.

Example Usage

Analyze VSAM dependencies in a core banking platform.

Review Checklist

- Access patterns captured?
- Dataset relationships captured?
- CRUD operations represented?
- Record structures captured?
- Metadata graph ready?

Future Enterprise Parsers

To support virtually any mainframe ecosystem, MIP should also include:

Prompt 11

CICS Parser Architecture

Prompt 12

MQ Parser Architecture

Prompt 13

IMS Parser Architecture

Prompt 14

REXX Parser Architecture

Prompt 15

Easytrieve Parser Architecture

Prompt 16

Scheduler Definition Parser

Prompt 17

Control Card Parser

Prompt 18

Assembler Parser

Prompt 19

PL/I Parser

Prompt 20

Natural/Adabas Parser

Part 2 Completion Criteria

Upon completion of these prompts, MIP should be capable of converting raw source code, operational definitions, and execution artifacts into structured metadata.

MIP must be able to answer:

- Which programs exist?
- Which jobs execute them?
- Which copybooks define data?
- Which DB2 tables are accessed?
- Which VSAM files are used?
- Which MQ queues are used?
- Which CICS transactions invoke programs?
- Which IMS transactions exist?
- Which dependencies exist?
- Which fields flow across systems?
- Which programs are root drivers?
- Which assets are modernization candidates?
- Which business capabilities are supported?

Only after parsing is complete should metadata modeling begin.

End of Part 2.